To complete **Stage 1: database read endpoints**, you need to update your `GET /tasks` and `GET /tasks/:id` route handlers. This will make them read records dynamically from your `tasks.db` file using safe, parameterized SQL queries.

Choose the code block below that matches your project lane.

Node.js Lane (Express + better-sqlite3)

Update the reading endpoints in your `server.js` file. Note that `better-sqlite3` uses `?` for placeholders, `.all()` to fetch arrays, and `.get()` to fetch a single row.

javascript

```
// ========================================
// Stage 1: GET /tasks (With Filtering)
// ========================================
app.get('/tasks', (req, res) => {
  const { done, search } = req.query;
  let query = 'SELECT * FROM tasks WHERE 1=1';
  const params = [];

  // Filter by completion status flag
  if (done !== undefined) {
    query += ' AND done = ?';
    params.push(done === 'true' ? 1 : 0);
  }

  // Filter by matching title keyword
  if (search !== undefined && search.trim() !== "") {
    query += ' AND LOWER(title) LIKE ?';
    params.push(`%${search.toLowerCase()}%`);
  }

  // Execute the safe query with variables passed separately
  const rows = db.prepare(query).all(...params);

  // Format SQLite integers (0 or 1) into clean JSON booleans
  const tasks = rows.map(row => ({
    id: row.id,
    title: row.title,
    done: Boolean(row.done)
```

```
  }));

  res.json(tasks);
});

// ========================================
// Stage 1: GET /tasks/:id (Single Selection)
// ========================================
app.get('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);

  // Parameterized query keeps database data safe from injection attacks
  const row = db.prepare('SELECT * FROM tasks WHERE id = ?').get(taskId);

  // Guard clause for unknown / missing record IDs
  if (!row) {
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }

  res.json({
    id: row.id,
    title: row.title,
    done: Boolean(row.done)
  });
});
```

Use code with caution.

🟡 Python Lane (FastAPI + sqlite3)

Update the endpoint paths in your `main.py` file. Using Python's built-in `sqlite3` driver requires explicitly fetching results from the database cursor object using `.fetchall()` or `.fetchone()`.

python

```
# ========================================
# Stage 1: GET /tasks (With Filtering)
# ========================================
@app.get("/tasks")
```

```
def read_tasks(done: Optional[bool] = Query(None), search: Optional[str] = Query(None)):
    conn = get_db_connection()
    cursor = conn.cursor()

    query = "SELECT * FROM tasks WHERE 1=1"
    params = []

    if done is not None:
        query += " AND done = ?"
        params.append(1 if done else 0)

    if search is not None and search.strip():
        query += " AND LOWER(title) LIKE ?"
        params.append(f"%{search.lower()}%")

    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()

    # Map row arrays into clean dictionaries with proper native booleans
    return [
        {"id": r["id"], "title": r["title"], "done": bool(r["done"])}
        for r in rows
    ]

# ========================================
# Stage 1: GET /tasks/{id} (Single Selection)
# ========================================
@app.get("/tasks/{id}")
def read_task(id: int):
    conn = get_db_connection()
    cursor = conn.cursor()

    # Passing params as a tuple handles string cleanups safely underneath
    cursor.execute("SELECT * FROM tasks WHERE id = ?", (id,))
    row = cursor.fetchone()
    conn.close()

    # Check if the query returned empty results
    if row is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail={"error": f"Task {id} not found"}
        )
```

```
    return {
        "id": row["id"],
        "title": row["title"],
        "done": bool(row["done"])
    }
```

Use code with caution.

🏁 Checkpoint Verification

Verify your API responses match your server's assigned port configuration (Node matches `3000`, Python matches `8000`):

1. **Verify Successful Read:**

   bash

   ```
   curl -i http://localhost:3000/tasks
   ```

   Use code with caution.

   Expected Outcome: HTTP status **200 OK** paired with a payload containing your three initial task objects.

2. **Verify Missing Item Request:**

   bash

   ```
   curl -i http://localhost:3000/tasks/999
   ```

   Use code with caution.

   Expected Outcome: HTTP status **404 Not Found** matching `{ "error": "Task 999 not found" }`.

💾 Git Commit

Run the commands below inside your workspace terminal structure to record Stage 1:

bash

```
git add server.js main.py
git commit -m "Stage 1: database read endpoints"
git push
```